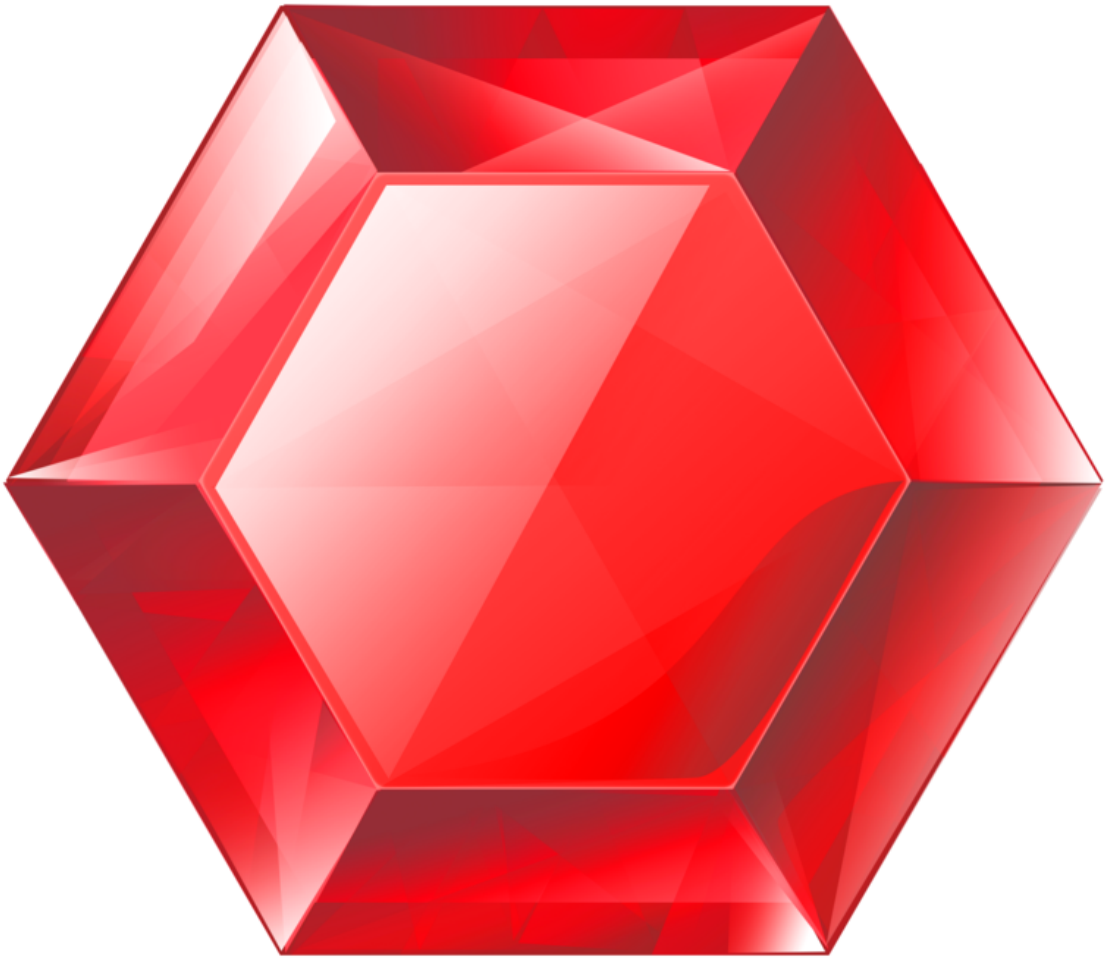
gemstone-rs Examples Guide

A tour of Rust examples and tooling workflows



Generated from the Markdown sources in gemstone-rs/docs.

Contents

Examples Guide

The `gemstone-rs` examples are split between runnable Cargo examples and the checked-in codegen sample under the repository-level `examples/` directory.

Common Setup

```
gemstone-rs env sample
gemstone-rs env write
gemstone-rs doctor --env-file .env.gemstone-rs
gemstone-rs --env-file .env.gemstone-rs browse dictionaries
gemstone-rs --env-file .env.gemstone-rs codegen check examples/codegen/gemstone-
rs.codegen
export GS_LIB=/opt/gemstone/product/lib
export GS_STONE=gs64stone
export GS_STONE_NAME=gs64stone
export GS_USERNAME=DataCurator
export GS_PASSWORD=swordfish
```

Run Cargo examples from the repository root:

```
cd /path/to/gemstone-rs
cargo run -p gemstone-rs --example quickstart
```

After installing the CLI, discover the same curated map without opening the repository docs:

```
gemstone-rs hello
gemstone-rs examples list
gemstone-rs examples show quickstart
gemstone-rs examples list --json
gemstone-rs examples map
gemstone-rs compare gemstone-py --status
gemstone-rs compare gemstone-py --scorecard
gemstone-rs compare gemstone-py --parity
gemstone-rs compare gemstone-py --gaps
gemstone-rs compare gemstone-py --next
gemstone-rs compare gemstone-py --totals
gemstone-rs compare gemstone-py --batches
gemstone-rs compare all --status
gemstone-rs compare all --scorecard
gemstone-rs compare all --parity
gemstone-rs compare all --next
gemstone-rs compare all --totals
gemstone-rs compare all --batches
gemstone-rs examples run codegen_preview --dry-run
```

```
gemstone-rs examples run axum_service --dry-run -- --routes
gemstone-rs examples run actix_service --dry-run -- --routes
gemstone-rs examples run python_native_adapter --dry-run
gemstone-rs examples run py_native_capabilities --dry-run
gemstone-rs examples run py_native_contract_fixture --dry-run
gemstone-rs examples run py_native_samples_fixture --dry-run
gemstone-rs examples run py_native_smoke_fixture --dry-run
gemstone-rs examples run py_native_migration_plan --dry-run
gemstone-rs examples run py_native_compatibility_fixture --dry-run
gemstone-rs examples run py_native_conformance_fixture --dry-run
gemstone-rs examples run py_native_handoff_bundle --dry-run
gemstone-rs examples run py_native_shared_core_gate --dry-run
gemstone-rs py-native capabilities --json
gemstone-rs py-native samples --json
gemstone-rs py-native migration --json
gemstone-rs py-native compatibility --json
gemstone-rs py-native conformance --json
gemstone-rs py-native handoff --json
gemstone-rs py-native check-all
gemstone-rs py-native check-all --json
gemstone-rs examples scaffold quickstart ./gemstone-rs-quickstart
gemstone-rs examples scaffold codegen_workflow ./gemstone-rs-codegen-workflow
gemstone-rs examples scaffold profile_codegen_workflow ./gemstone-rs-profile-codegen
gemstone-rs examples scaffold generated_wrapper_app ./gemstone-rs-generated-wrapper
gemstone-rs examples scaffold py_native_pyo3_adapter ./gemstone-py-native-starter
gemstone-rs examples scaffold axum_service ./gemstone-rs-axum-service
gemstone-rs examples scaffold actix_service ./gemstone-rs-actix-service
```

From a source checkout, `gemstone-rs examples run <name>` can launch the selected Cargo example or CLI-backed example directly. That includes the py-native fixture checks, so `examples list`, VS Code, and CI all point at the same adapter contract workflows. Use `--dry-run` when you only want to verify the command that would run. From an installed CLI, `gemstone-rs examples scaffold <name> [path]` writes a standalone Cargo project from embedded templates. Current scaffold templates include `quickstart`, `browser`, `bridge_root_mapping`, `derive_mapping`, `codegen_preview`, `codegen_workflow`, `codegen_discover`, `codegen_discover_mapping`, `profile_codegen_workflow`, `generated_wrapper_app`, `generated_mapping_app`, `http_service`, `session_worker_pool`, `py_native_pyo3_adapter`, `axum_service`, and `actix_service`; checked-in examples also include `bridge_value_inspection` for dynamic nested BridgeRoot read-back and `python_native_adapter` for the future `gemstone-py-native` wrapper contract. The `py_native_pyo3_adapter` scaffold writes a `pyproject.toml`, PyO3 `src/lib.rs`, and Python smoke tests that wrap the dependency-free `gemstone_rs::py_native` contract, including `capabilities_json`, `samples_json`, `smoke_dry_run_json`, `migration_json`, `compatibility_json`, `conformance_json`, and `handoff_json`, plus direct `NativeSession` methods for eval, execute, resolve, value-to-OOP conversion, perform, strings, symbols, globals, export-set retention, and transactions. `eval_json` and `perform_json` return the stable `PyNativeValue` JSON shape for Python package code to decode. It also writes `python/gemstone_py_native_compat.py`, which wraps raw native OOP returns in `OopHandle` through `NativeCompatibilitySession` so package code can preserve existing Python return behavior while typed helpers and value conversion remain opt-in. It uses PyO3 0.28 so the

generated starter remains compatible with current Python 3.14 interpreters, and `maturin` enables the `extension-module` Cargo feature only for Python extension builds. Source checkout verification also compiles and runs the scaffold against the local Rust core:

```
python3 scripts/check_py_native_pyo3_scaffold.py
```

The py-native examples also include a checked-in value/error samples fixture:

```
gemstone-rs py-native check-samples examples/py-native/gemstone-rs.py-native-
samples.json
gemstone-rs py-native migration --json
```

That fixture gives wrapper CI concrete payloads for `nil`, booleans, small integers, characters, strings, symbols, OOPs, and structured errors. The `py-native migration --json` report tracks the `gemstone-py-native` shared-core path: wrapping `PyNativeSession`, preserving existing Python return behavior, keeping the live backend smoke green, and verifying published wheels from TestPyPI/PyPI installs. `py-native compatibility --json` and the checked-in `examples/py-native/gemstone-rs.py-native-compat.json` fixture document the generated compatibility shim: `NativeCompatibilitySession`, `OopHandle`, and the method-by-method mapping from Python-facing calls to raw native adapter calls. `py-native conformance --json` and `examples/py-native/gemstone-rs.py-native-conformance.json` add the end-to-end wrapper target: extension module functions, raw `NativeSession` methods, compatibility shim methods, fixture paths, and scaffold files. `py-native handoff --json` and `examples/py-native/gemstone-rs.py-native-handoff.json` add the final downstream handoff bundle: every fixture path, schema, regeneration command, validation command, and acceptance check that `gemstone-py-native` needs before using the Rust core as its native backend. `py-native check-all` is the downstream shared-core gate. It validates the capabilities, samples, smoke, compatibility, conformance, and handoff fixtures together, and `--json` gives CI or VS Code a single status report.

Aliases include `bridge`, `mapping`, `derive`, `codegen`, `discover`, `profiles`, `wrapper`, `framework`, `axum`, `actix`, and `http`. Scaffolds can include supporting project files; `profile_codegen_workflow` writes both `gemstone-rs.codegen` and `gemstone-rs.codegen-profiles.json`. `gemstone-rs examples map` is the Rust equivalent of `gemstone-examples plan3-map`: it groups crates, examples, docs, and gemstone-py reference points by feature stream. The same content is maintained in feature-map.md.

`gemstone-rs hello` is the no-live equivalent of `gemstone-examples hello`. Use it before configuring GemStone credentials when you only want to prove the CLI binary is installed and runnable.

Example Map

Feature	Command or path	What it demonstrates
Hello CLI	`gemstone-rs hello`	Verifies the installed CLI without GemStone credentials.
Scaffold quickstart	`gemstone-rs examples scaffold quickstart ./gemstone-rs-quickstart`	Creates a standalone Cargo quickstart project from the installed CLI.
Scaffold browser	`gemstone-rs examples scaffold browser ./gemstone-rs-browser`	Creates a standalone class-browser project from the installed CLI.
Scaffold BridgeRoot mapping	`gemstone-rs examples scaffold bridge_root_mapping ./gemstone-rs-bridge-root-mapping`	Creates a standalone BridgeRoot mapping project from the installed CLI.
Scaffold derive mapping	`gemstone-rs examples scaffold derive_mapping ./gemstone-rs-derive-mapping`	Creates a standalone derive-mapping project from the installed CLI.
Scaffold codegen preview	`gemstone-rs examples scaffold codegen_preview ./gemstone-rs-codegen-preview`	Creates a standalone no-live codegen preview project from the installed CLI.
Scaffold codegen workflow	`gemstone-rs examples scaffold codegen_workflow ./gemstone-rs-codegen-workflow`	Creates a standalone no-live codegen preview/diff/check/generate project from the installed CLI.
Scaffold codegen discovery	`gemstone-rs examples scaffold codegen_discover ./gemstone-rs-codegen-discover`	Creates a standalone live discovery project from the installed CLI.
Scaffold mapping discovery	`gemstone-rs examples scaffold codegen_discover_mapping ./gemstone-rs-codegen-discover-mapping`	Creates a standalone live mapping discovery project from the installed CLI.
Scaffold profile codegen	`gemstone-rs examples scaffold profile_codegen_workflow ./gemstone-rs-profile-codegen`	Creates a standalone profile-driven codegen project with config and profile files.
Scaffold generated wrapper	`gemstone-rs examples scaffold generated_wrapper_app ./gemstone-rs-generated-wrapper`	Creates a standalone generated-style wrapper app from the installed CLI.

Feature	Command or path	What it demonstrates
Scaffold generated mapping	`gemstone-rs examples scaffold generated_mapping_app ./gemstone-rs-generated-mapping`	Creates a standalone generated-style BridgeMapped app from the installed CLI.
Scaffold HTTP service	`gemstone-rs examples scaffold http_service ./gemstone-rs-http-service`	Creates a standalone Rust HTTP health-service project from the installed CLI.
Scaffold worker pool	`gemstone-rs examples scaffold session_worker_pool ./gemstone-rs-worker-pool`	Creates a standalone bounded SessionWorkerPool project from the installed CLI.
Scaffold PyO3 adapter	`gemstone-rs examples scaffold py_native_pyo3_adapter ./gemstone-py-native-starter`	Creates a starter `gemstone-py-native` PyO3 crate over `gemstone_rs::py_native`.
Scaffold Axum service	`gemstone-rs examples scaffold axum_service ./gemstone-rs-axum-service`	Creates a standalone Axum health-service project from the installed CLI.
Scaffold Actix service	`gemstone-rs examples scaffold actix_service ./gemstone-rs-actix-service`	Creates a standalone Actix Web health-service project from the installed CLI.
First login	`cargo run -p gemstone-rs --example hello_gemstone`	Reads env config, logs in, prints a session id, and evaluates `3 + 4`.
Quickstart	`cargo run -p gemstone-rs --example quickstart`	Eval, `global_put`, `global_get`, string fetch, cleanup.
Eval only	`cargo run -p gemstone-rs --example eval`	Minimal `Session::eval` shape.
Browser API	`cargo run -p gemstone-rs --example browser`	Dictionaries, protocols, methods, and source.
Live smoke cookbook	`cargo run -p gemstone-rs --example live_smoke_cookbook`	Login, eval, global round-trip, perform, and transaction checks in one run.
Transactions	`cargo run -p gemstone-rs --example transactions`	Commit-on-success and abort-on-error transaction wrapper.

Feature	Command or path	What it demonstrates
Session worker	`cargo run -p gemstone-rs --example session_worker`	Dedicated-thread `SessionWorker` for web services and async runtimes.
Session worker pool	`cargo run -p gemstone-rs --example session_worker_pool`	Bounded round-robin pool of dedicated GemStone session workers.
Async worker facade	`cargo run -p gemstone-rs --example async_worker`	Awaitable `SessionWorkerPool` calls for async runtimes without moving `Session` across threads.
Python native adapter	`gemstone-rs py-native smoke --dry-run`; `cargo run -p gemstone-rs --example python_native_adapter -- --dry-run`	Dependency-free `py_native` contract used by the `gemstone-py-native` PyO3 bridge and smoke tests.
Python native contract fixtures	`gemstone-rs py-native check examples/py-native/gemstone-rs.py-native.json`; `gemstone-rs py-native check-smoke examples/py-native/gemstone-rs.py-native-smoke.json`	Checked-in capability and smoke samples for wrapper CI and editor tooling.
Python native value/error samples	`gemstone-rs py-native check-samples examples/py-native/gemstone-rs.py-native-samples.json`	Concrete value and error payload samples for Python wrapper translation tests.
Python native migration plan	`gemstone-rs py-native migration --json`; `gemstone-rs examples run py_native_migration_plan --dry-run`	Machine-readable checklist for completing the gemstone-py-native shared-core migration.
Python native compatibility fixture	`gemstone-rs py-native check-compat examples/py-native/gemstone-rs.py-native-compat.json`; `gemstone-rs examples run py_native_compatibility_fixture --dry-run`	Checks the Python package-layer return policy and shim method mapping.
Python native conformance fixture	`gemstone-rs py-native check-conformance examples/py-native/gemstone-rs.py-native-conformance.json`; `gemstone-rs examples run py_native_conformance_fixture --dry-run`	Checks the PyO3 module/session/shim surface that a real `gemstone-py-native` wrapper should expose.

Feature	Command or path	What it demonstrates
Python native handoff bundle	`gemstone-rs py-native check-handoff examples/py-native/gemstone-rs.py-native-handoff.json`; `gemstone-rs examples run py_native_handoff_bundle --dry-run`	Checks the downstream `gemstone-py-native` handoff manifest across contract, smoke, compatibility, conformance, and acceptance criteria.
Python native shared-core gate	`gemstone-rs py-native check-all`; `gemstone-rs examples run py_native_shared_core_gate --dry-run`	Checks every checked-in py-native fixture in one downstream CI gate.
Python native examples runner	`gemstone-rs examples run py_native_capabilities --dry-run`; `gemstone-rs examples run py_native_contract_fixture --dry-run`; `gemstone-rs examples run py_native_smoke_fixture --dry-run`; `gemstone-rs examples run py_native_migration_plan --dry-run`; `gemstone-rs examples run py_native_compatibility_fixture --dry-run`; `gemstone-rs examples run py_native_conformance_fixture --dry-run`; `gemstone-rs examples run py_native_handoff_bundle --dry-run`; `gemstone-rs examples run py_native_shared_core_gate --dry-run`	Exposes py-native fixture workflows through the same examples catalog as Cargo examples.
OOP/value conversion	`cargo run -p gemstone-rs --example oop_values`	`Value`, `Oop`, strings, symbols, and export-set retention.
BridgeRoot mapping	`cargo run -p gemstone-rs --example bridge_root_mapping`	MagLev-style bridge-root storage with explicit `BridgeValue` mapping.
Derive mapping	`cargo run -p gemstone-rs --example derive_mapping`	`#[derive(BridgeMapped)]`, symbol keys, nested structs, vectors, maps, and BridgeRoot transactions.
BridgeValue inspection	`cargo run -p gemstone-rs --example bridge_value_inspection`	Reads nested BridgeRoot dictionaries and arrays back as dynamic `BridgeValue` trees with shape reports.

Feature	Command or path	What it demonstrates
Bridge mapping preview	`cargo run -p gemstone-rs --example bridge_mapping_preview`	Infers a starter `BridgeMapped` codegen config from a nested `BridgeValue` tree.
Offline codegen	`cargo run -p gemstone-rs --example codegen_preview`	Generates wrappers from the sample config without a live stone.
Codegen workflow	`cargo run -p gemstone-rs --example codegen_workflow`	Writes config, previews, diffs, checks, generates, and verifies a clean diff.
Codegen discovery	`cargo run -p gemstone-rs --example codegen_discover`	Connects to a live stone and discovers a starter config for `Object`.
Mapping discovery	`cargo run -p gemstone-rs --example codegen_discover_mapping`	Connects to a live stone and proposes a `BridgeMapped` config.
Generated wrapper app	`cargo run -p gemstone-rs --example generated_wrapper_app`	Uses checked-in generated wrappers to call `Object>>printString`.
Generated mapping app	`cargo run -p gemstone-rs --example generated_mapping_app`	Uses codegen-created `BridgeMapped` structs with `BridgeRoot`.
Generated wrapper compile check	`cargo test --manifest-path examples/codegen-wrapper-check/Cargo.toml`	Imports the checked-in generated wrappers as a separate crate and runs generated metadata tests.
Codegen files	`examples/codegen/`	Config, generated wrappers, check/diff/generate workflow.
Explorer tooling	`examples/tooling/explorer.md`	Local explorer startup and endpoint checks.
VS Code tooling	`examples/tooling/vscode-workbench.md`	Sidebar browsing, codegen actions, and explorer launch.
CLI browser walkthrough	`examples/tooling/cli-browser-walkthrough.md`	Terminal-only browse workflow.
HTTP service		

Feature	Command or path	What it demonstrates
	`cargo run -p gemstone-rs --example http_service -- --routes`	Standard-library web service with `/`, `/health/local`, and `/health/gemstone`.
Axum service	`cargo run --manifest-path examples/axum-service/Cargo.toml -- --routes`	Checked Axum route shape with `gemstone-rs-axum`, `SessionWorkerPool`, and shared `gemstone_rs::web` health responses.
Actix service	`cargo run --manifest-path examples/actix-service/Cargo.toml -- --routes`	Checked Actix route shape with `gemstone-rs-actix`, `SessionWorkerPool`, and shared `gemstone_rs::web` health responses.

The Axum and Actix services use the graceful health-pool startup path. They can start with missing credentials, serve `/` and `/health/local`, and return a `503` JSON error from `/health/gemstone` until the stone is configured. Verify the route contract with:

```
python3 scripts/framework_route_smoke.py
scripts/live_smoke.sh --dry-run
scripts/live_smoke.sh
```

The same smoke check asserts diagnostic and request-trace headers from the adapters: `x-gemstone-rs-adapter`, `x-gemstone-rs-route`, `x-gemstone-rs-request-id`, `x-gemstone-rs-request-method`, and `x-gemstone-rs-request-path`. It also asserts the lifecycle headers `x-gemstone-rs-request-lifecycle: received,handled` and `x-gemstone-rs-request-duration-us`, which give tests and local proxies a framework-neutral way to confirm the packaged handler ran. The checked services also add `x-gemstone-rs-example-middleware: axum` or `x-gemstone-rs-example-middleware: actix`, `x-gemstone-rs-service`, `x-gemstone-rs-service-version`, `cache-control: no-store`, and `x-content-type-options: nosniff`. The smoke script asserts those headers so application middleware and a small production-style cache/security policy stay covered. Use `scripts/live_smoke.sh` when GemStone credentials are available and `/health/gemstone` should be required to return `{"result":7}` as part of the same live lane that runs Rust tests and live examples.

Suggested Learning Order

1. `gemstone-rs hello`
2. `hello_gemstone`
3. `quickstart`
4. `browser`

5. `live_smoke_cookbook`
6. `transactions`
7. `session_worker`
8. `session_worker_pool`
9. `async_worker`
10. `oop_values`
11. `bridge_root_mapping`
12. `derive_mapping`
13. `bridge_value_inspection`
14. `codegen_preview`
15. `codegen_workflow`
16. `generated_wrapper_app`
17. `generated_mapping_app`
18. `codegen_discover`
19. `codegen_discover_mapping`
20. `examples/codegen/`
21. `examples/tooling/cli-browser-walkthrough.md`
22. `examples/tooling/explorer.md`
23. `examples/tooling/vscode-workbench.md`
24. `http_service`
25. `examples/axum-service/`
26. `examples/actix-service/`

Expected Output

Live examples need GemStone credentials and a reachable stone. If the environment is configured correctly, these are the important lines to look for:

```
$ cargo run -p gemstone-rs --example hello_gemstone
session id: <number>
3 + 4 => SmallInt(7)

$ cargo run -p gemstone-rs --example quickstart
GemStone eval ok: SmallInt(7)
GemStoneRsQuickstart: hello from gemstone-rs quickstart

$ cargo run -p gemstone-rs --example generated_wrapper_app
generated wrapper printString: 7

$ cargo run -p gemstone-rs --example live_smoke_cookbook
login ok: session <number>
eval ok: SmallInt(7)
global round-trip ok
perform ok: 7
transaction commit/abort ok

$ cargo run -p gemstone-rs --example bridge_root_mapping
```

```
bridge root: GemStoneRsBridgeRoot
MyTestDict OOP: <number>
loaded payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP", labels:
{"source": "manual"} }
loaded status: ready
loaded amount: 100
loaded approved: true
loaded tags: ["priority", "demo"]
loaded note: Some("front desk")
loaded labels: {"source": "manual"}
loaded symbol labels: {"source": "manual"}

$ cargo run -p gemstone-rs --example derive_mapping
derived mapped payload: BookingDraft { amount: 100, customer: CustomerDraft { name:
"Tariq" }, tags: ["priority", "demo"], labels: {"source": "derive"}, note: None }

$ cargo run -p gemstone-rs --example bridge_value_inspection
dynamic BridgeValue: Dictionary({"customer": Dictionary(...), "items": Array(...),
"note": Nil, "state": Symbol("ready")})
bridge root identity: <number>
bridge root key count: <number>

$ cargo run -p gemstone-rs --example generated_mapping_app
generated mapped payload: BookingDraft { name: "Tariq", amount: 100, currency: "GBP",
tags: ["priority", "demo"], labels: {"source": "generated"}, note: Some("window
seat") }

$ cargo run -p gemstone-rs --example http_service -- --routes
gemstone-rs HTTP service example
  GET /
  GET /health/local
  GET /health/gemstone
```

Offline examples should run without GemStone:

```
$ cargo run -p gemstone-rs --example codegen_workflow
before generate: exists=false up_to_date=false diff_bytes=<number>
after generate: exists=true up_to_date=true
diff after generate: clean
```

The checked-in codegen sample also demonstrates typed arguments. A config line such as `method = Object>>perform: | args=selector:Symbol` generates a Rust method that accepts `selector: impl AsRef<str>` and creates the GemStone symbol before dispatch. For application wrappers, use `SmallInt`, `String`, `Symbol`, `Bool`, or the default `Oop` depending on how much conversion you want at the generated boundary. Typed returns cover the same common value shapes: `return=Symbol` fetches the GemStone Symbol as a Rust `String`, while `return=SmallInt`, `return=Bool`, `return=String`, and `return=Oop` narrow the result at the wrapper boundary.

CLI Equivalents

The CLI gives you the same live checks without compiling examples:

```
cargo install gemstone-rs-cli
gemstone-rs doctor
gemstone-rs doctor --live
gemstone-rs doctor --strict
gemstone-rs doctor --json
gemstone-rs eval --env-file .env.gemstone-rs "3 + 4"
gemstone-rs browse dictionaries
gemstone-rs browse classes UserGlobals
gemstone-rs browse protocols Object
gemstone-rs browse methods Object "-- all --"
gemstone-rs browse source Object printString
gemstone-rs inspect oop 20
gemstone-rs bridge root
gemstone-rs bridge keys
gemstone-rs bridge put-string WorkbenchDraft "hello from Rust"
gemstone-rs bridge put-symbol WorkbenchState ready
gemstone-rs bridge put-smallint WorkbenchCount 7
gemstone-rs bridge put-bool WorkbenchReady true
gemstone-rs bridge remove WorkbenchDraft
gemstone-rs bridge sample-config BookingDraft
gemstone-rs codegen explain examples/codegen/gemstone-rs.codegen
gemstone-rs codegen explain --json examples/codegen/gemstone-rs.codegen
gemstone-rs codegen explain-profile --json default examples/codegen/gemstone-
rs.codegen-profiles.json
gemstone-rs compare gemstone-py --status
gemstone-rs compare gemstone-py --scorecard
gemstone-rs compare gemstone-py --parity
gemstone-rs compare gemstone-py --gaps
gemstone-rs compare gemstone-py --next
gemstone-rs compare gemstone-py --totals
gemstone-rs compare gemstone-py --batches
gemstone-rs compare all --status
gemstone-rs compare all --scorecard
gemstone-rs compare all --parity
gemstone-rs compare all --next
gemstone-rs compare all --totals
gemstone-rs compare all --batches
gemstone-rs examples scaffold quickstart ./gemstone-rs-quickstart
gemstone-rs examples scaffold codegen_workflow ./gemstone-rs-codegen-workflow
gemstone-rs examples scaffold profile_codegen_workflow ./gemstone-rs-profile-codegen
gemstone-rs examples scaffold generated_wrapper_app ./gemstone-rs-generated-wrapper
gemstone-rs examples scaffold py_native_pyo3_adapter ./gemstone-py-native-starter
gemstone-rs examples scaffold axum_service ./gemstone-rs-axum-service
gemstone-rs examples scaffold actix_service ./gemstone-rs-actix-service
gemstone-rs examples run axum_service --dry-run -- --routes
gemstone-rs examples run actix_service --dry-run -- --routes
cargo test --manifest-path examples/codegen-wrapper-check/Cargo.toml
```

Codegen Workflow

```
cargo run -p gemstone-rs-cli -- codegen init examples/codegen/demo.codegen
cargo run -p gemstone-rs-cli -- codegen preview examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen diff examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen check examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen generate examples/codegen/gemstone-rs.codegen
cargo run -p gemstone-rs-cli -- codegen discover-mapping examples/codegen/
mapping.codegen BookingDraft Object
```

Use the checked-in explorer profile sample when you want a repeatable browser workflow for codegen:

```
cat examples/codegen/gemstone-rs.codegen-profiles.json
gemstone-rs-explorer --port 8787 --codegen-root .
```

In the explorer, click `Load Project Profiles` with `examples/codegen/gemstone-rs.codegen-profiles.json` as the project profile file, then select `default`, `object-wrapper`, or `bridge-mapping`.

Validate profile files before committing them:

```
cargo run -p gemstone-rs-cli -- profile validate examples/codegen/gemstone-
rs.codegen-profiles.json
cargo run -p gemstone-rs-cli -- profile check examples/codegen/gemstone-rs.codegen-
profiles.json
cargo run -p gemstone-rs-cli -- codegen explain-profile --json default examples/
codegen/gemstone-rs.codegen-profiles.json
```

Generate a starter config from a live stone:

```
cargo run -p gemstone-rs-cli -- codegen discover examples/codegen/discovered.codegen
Object
cargo run -p gemstone-rs --example codegen_discover_mapping
```

Run the generated wrapper example after checking the generated file into the repository:

```
cargo run -p gemstone-rs --example generated_wrapper_app
```

VS Code

Install the workbench from:

```
https://marketplace.visualstudio.com/items?itemName=unicompute.gemstone-rs-workbench
```

Use the GemStone RS sidebar to browse dictionaries, classes, protocols, and methods. The same sidebar exposes Codegen Discover, Preview, Diff, Check, Generate, profile-driven codegen actions, Generate Mapping Config, Preview BridgeRoot, List BridgeRoot Keys, Put BridgeRoot String, Put BridgeRoot Symbol, Put BridgeRoot SmallInt, Put BridgeRoot Bool, Remove BridgeRoot Key, Run Generated Mapping Example, Load Project Profiles, Save Project Profiles, Export Codegen Profile, Show Sample Project Profiles, Create Project Profiles, Validate Project Profiles, List Project Profiles, Show Project Profile, Resolve Project Profile, Check Project Profiles, and Open Docs actions.

Later Examples

These are useful, but should wait until the corresponding APIs are stable:

- a local explorer workflow with screenshots
- broader framework coverage and richer real-application middleware examples
- a richer class browser walkthrough with captured output

This PDF was generated locally from `docs/`. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.